

doip tp过程以及其中的dcms分析再重新完整整理下

- **Read** UdsTp.c (lines 374-453)
- Thought for 1s
- **Read** UdsTp.c (lines 457-536)
- Thought for 1s
- Now let me also check the `dcos_uds_service_client_init` to see how the SOC-side client connects to the cross-domain service.
- **Read** `uds_related_service.c` (lines 796-875)
- Thought for 2s >

- **Bash** Find example local DID handler implementation

```
IN  grep -rn "DataServices_0xF187_FAW_Part_Number_16Bytes_ReadData" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/ -l 2>/dev/null

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/dji_rte.c
    /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.c
    /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.h
```

- Thought for 1s
- **Bash** Show local DID handler implementation example

```
IN  grep -n "DataServices_0xF187_FAW_Part_Number_16Bytes_ReadData" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam

OUT 807:Std_ReturnType DataServices_0xF187_FAW_Part_Number_16Bytes_ReadData(Dcm_OpStatusType OpStatus, P2VAR(uint8, AUTOMATIC, DCM_VAR_NOINIT) Data)
808-{
809-    Std_ReturnType ret = E_OK;
    ...
```

- Thought for 7s >
- 现在所有关键代码都读完了，下面是完整准确的整理。

DoIP TP 全过程与 DCMS 架构分析

一、初始化链路

当 `dji_doip_service_ipv4` 进程启动时，执行以下初始化顺序 (`appl_ipv4_main.c:104-198`):

1. `uds_minidcos_service_init("udsonip_ipv4_proxy", 5)`
 - └ 创建 20MB MiniDCOS 共享内存实例
 - └ 注册 5 个 module (`up_server_module_proxy0~4`)
2. `dji_rte_config(&rte_config)`
 - └ 把 `dji_rte.c` 中的函数指针绑定到全局 `rte_config` 结构体
3. `dji_udsonip_init()`
 - └ 初始化 AUTOSAR BSW 全栈: SoAd → DoIP → PduR → DCM → Dem
4. `uds_related_service_init_async(minidcos_init_fin_event_ipv4)`
 - └ `dcos_uds_service_client_init()`:
 - `minidcos_if_reg_module3()` → 再注册一个 module
 - `minidcos_if_get_service_with_cbk()` → 为 `service_param[]` 中每一项注册 CLIENT 回调 `call_uds_service_cb`
 - `event_task_create("uds_cb_monitor")` → 创建超时监控线程
 - └ 回调 `minidcos_init_fin_event_ipv4()`:
 - `dcms_mcu_topic_register("uds_v4_app", DCMS_MCU_UDA_TOPIC_CONFIG)` → 批量注册所有 MiniDCOS topic
5. `CrossDomainUdsInit()`
 - └ `minidcos_service_server_reg(SERVICE_CROSS_UDS_RESP, cross_diag_data_response_cb)` → SOC 注册为跨域响应的 SERVER
 - └ `event_task_create("uds_cb_monitor_cross_domain")` → 跨域超时监控线程
6. `event_task_create("dudsopip", dji_udsopip_task_handler, prio=63)`
 - └ 创建主循环线程 → OS_Tick_Task → BSW 各模块 MainFunction

二、两条实际 UDS 处理路径

`rte_config` 函数指针表 (`dji_rte.c:20-127`) 将不同服务的处理分配到了两条路径:

```
DCM 收到 UDS 请求
┌
├ 0x10, 0x11, 0x27, 0x28, 0x2E, 0x31, 0x34-0x38 → DcmDslServiceRequestSupplierNotification_Indication (pre-check only)
├
├ 正式处理由 DiagSwc_Callback.c 分发 (自动生成的 RTE dispatch):
├
├ 【路径A: 本地处理】 DID/RID callbacks
├ SID 0x22 (Read DID): DID_Read_ScVGRop.xxx → DataServices_xxx_ReadData()
├
```

```
→ 直接从本地内存/NvM读取数据，无网络通信
→ 例：DataServices_0xF187_FAW_PartNumber_16Bytes_ReadData(OpStatus, Data)
    直接 memcpy(Data, HardAndPart_Version_Info, sizeof(g_part_number))

SID 0x2E (Write DID): DID_Write_ScvGrop.xxx → DataServices_xxx_WriteData()
→ 直接写本地，无网络通信

SID 0x31 (Routine): RoutineControl_ScvGrop.xxx → UpdateAdapter_xxx / EOL_xxx
→ 本地处理，无网络通信

SID 0x34-0x38 (Download/Upload): → UdsTransfer_xxx()
→ 本地文件传输处理

【路径B：跨域到MCU】 Service-level callbacks
SID 0x14 (Clear DTC): → DiagnosticService_0x14..._Proxy()
SID 0x19 (Read DTC): → DiagnosticService_0x19..._Proxy()
SID 0x28 (Com Control): → DiagnosticService_0x28..._Proxy()
SID 0x85 (DTC Setting): → DiagnosticService_0x85..._Proxy()
↓
cross_uds_req_msg_context() / cross_uds_req_fixlen()
↓
call_uds_service_async_cross_domain(SERVICE_CROSS_UDS_REQ, ...)
↓
MiniDCOS → MCU
```

三、路径B——跨域到MCU的完整TP时序

3.1 核心数据结构

两个 MiniDCOS service name 形成了双向通道：

```
// dsar_plat_bt/diag/uds_related_service.h:69-70
#define SERVICE_CROSS_UDS_REQ "/sys/cross_domain_req/uds" // SOC-MCU 请求
#define SERVICE_CROSS_UDS_RESP "/sys/cross_domain_resq/uds" // SOC-MCU 响应
```

SOC 和 MCU 两端的角色：

方向	SOC 角色	MCU 角色	MiniDCOS Topic
请求 SOC → MCU	CLIENT (minidcos_if_get_service_with_cbk)	SERVER (dcms_mcu_topic_setup_callback)	/sys/cross_domain_req/uds
响应 SOC ← MCU	SERVER (minidcos_service_server_reg)	CLIENT (dcms_mcu_topic_send)	/sys/cross_domain_resq/uds

3.2 异步请求+轮询状态机 (SOC 侧)

没有内部循环。状态机每一步都由 DCM 主循环驱动（每 ~5-10ms 调用一次）：

```
Tick 1: OpStatus = DCM_INITIAL
DiagnosticService_0x14..._Proxy()
└─ cross_uds_req_msg_context(INITIAL, ...)
    └─ call_uds_service_async_cross_domain(SERVICE_CROSS_UDS_REQ, req, len, timeout=5s)
        └─ reset_resp_data_cross_domain() // 清空响应buffer
        └─ minidcos_if_call_service(client_handle, req_msg) // 异步发送到MCU
            └─ pthread_cond_signal(start_cond) // 唤醒超时监控线程
        └─ return DCM_E_PENDING // 告诉DCM：还没完，下个周期再来

Tick 2..N: OpStatus = DCM_PENDING
DiagnosticService_0x14..._Proxy()
└─ if (pending_count++ % 20 == 0): // 每100ms检查一次
    cross_uds_req_msg_context(PENDING, ...)
    └─ get_resp_data_cross_domain(&resp_data, ...) // 检查响应buffer
        └─ UDS UPDATE RESULT_OK → 有响应
            └─ SID*0x40 == resp[0] → 正确响应 → Dcm_UtiProvideResDataAsUN() → return E_OK
            └─ 0x7F + 0x78 → pending → re-send request → return DCM_E_PENDING
            └─ 0x7F + NRC → 负响应 → *ErrorCode = NRC → return E_NOT_OK
        └─ UDS UPDATE RESULT_WAITING → return DCM_E_PENDING
    └─ else → return E_NOT_OK
else if (pending_count > 3000): // 3000*5ms=15s 超时
    return E_NOT_OK
```

3.3 响应接收：两条 MiniDCOS 回调路径 (SOC 侧)

MCU 的响应通过 MiniDCOS 回到 SOC，触发两个回调之一（取决于 MiniDCOS 内部实现路径）：

路径1: call_uds_service_cb (client注册时的回调，由 minidcos_if_get_service_with_cbk 设置)

```
// uds_related_service.c:734
call_uds_service_cb(mid, resp_msg, param)
└─ minidcos_if_msg_get_entity(resp_msg) → 获取响应数据
└─ set_resp_data(resp_msg.data, len) → 写入 resp_data buffer (互斥锁保护)
└─ pthread_cond_signal(exec_cond) → 唤醒 uds_cb_monitor 等待线程
```

路径2: cross_diag_data_response_cb (SOC SERVER 注册的回调)

```
// uds_related_service.c:766
cross_diag_data_response_cb(req, req_size, resp, resp_buf_size, resp_out_len)
└─ set_resp_data_cross_domain(req, req_size) → 写入 resp_data_cross_domain buffer
└─ pthread_cond_signal(exec_cond) → 唤醒 uds_cb_monitor 等待线程
```


pthread_cond_signal(exec_cond) → 唤醒 uds_cb_monitor_cross_domain 等待线程

路径1的 set_resp_data 写入 resp_data；路径2的 set_resp_data_cross_domain 写入 resp_data_cross_domain。get_resp_data_cross_domain 仅读 resp_data_cross_domain。但 call_uds_service_cb 中有特殊判断：如果收到 0xFF 0xFF 0xFF（这是 MCU 的 ack 确认），就忽略不管，等 cross_diag_data_response_cb 的正式响应。

3.4 MCU 侧：接收请求 + 返回响应

```
// DiagApp.c:396 - MCU 启动时注册
dcms_mcu_topic_setup_callback(DCMS_TOPIC_DIAG_CROS_UDS_REQ,
    cross_diag_request_cb, ...); // MCU作为SERVER接收SOC请求
dcms_mcu_service_client_setup_resp_callback(DCMS_TOPIC_DIAG_CROS_UDS_RESP,
    cross_diag_response_cb); // MCU作为CLIENT发送响应的回执

// Diag_CrossDiag.c:249 - SOC请求到达时
cross_diag_request_cb(data, len)
├─ 0x28/0x85 → 本地直接处理 (Com Control、DTC Setting)
├─ 其他SID → 注入AUTOSAR DCM:
│   ├── Dcm_StartOfReception() → Dcm_CopyRxData() → Dcm_TpRxIndication(E_OK)
│   ├── → DCM 内部处理 → DCM 调用 RTE callbacks
│   └─ → 处理完成后 DCM 调用 PduR_UpTransmit doip_cross_diag()
│       ├── memcpy(doip_cross_diag_resp.data, info->SduDataPtr, info->SduLength)
│       └─ 检查并发送响应:
│           ├── if (doip_cross_diag_resp.data.len > 0):
│           │   ├── dcms_mcu_topic_send(DCMS_TOPIC_DIAG_CROS_UDS_RESP,
│           │   │   doip_cross_diag_resp.data, doip_cross_diag_resp.data.len)
│           │   └─ MiniDCOS → SOC
│           └─ // 发送ACK (0xFF 0xFF 0xFF) 确认已收到请求
│               ├── dcms_mcu_service_server_set_ackdata(DCMS_TOPIC_DIAG_CROS_UDS_REQ,
│               │   doip_cross_diag_request_ack, sizeof(doip_cross_diag_request_ack))
```

3.5 超时监控 (SOC 侧)

```
// uds_related_service.c:685
uds_cb_monitor_cross_domain() // 独立线程, prio=63
while(true):
    pthread_cond_wait(start_cond) // 等待call_uds_service_async_cross_domain唤醒
    pthread_cond_timedwait(exec_cond, timeout_s) // 等待响应或超时
    if (ETIMEDOUT):
        set_resp_data_cross_domain({0x7F, 0x00, 0x70}) // 超时-注入假NRC 0x70
```

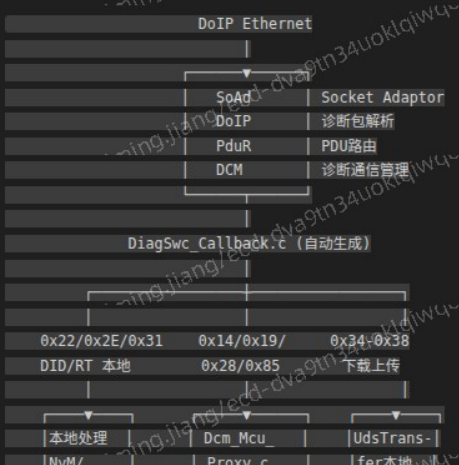
四、UdsTp.c —— 未被使用的 DCOS 方案

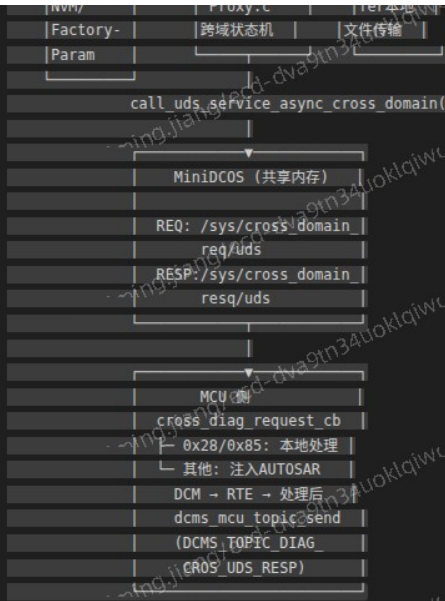
UdsTp.c 中的函数 (UdsTp_Diag_14Svc、UdsTp_Diag_22Svc 等) 在当前产品中完全没有被调用。它们是另一套 UDS 传输层的实现，和 MiniDCOS 跨域方案是平行替代：

方案对比：

UdsTp (DCOS方案, 未启用)	当前方案 (MiniDCOS跨域)
底层: DCOS (DDS)	底层: MiniDCOS (共享内存)
进程: dji_ad_app (主进程)	进程: dji_doip_service_ipv4
发送: dcms_mcu_topic_send()	发送: minidcos_if_call_service()
接收: dcms_mcu_topic_setup_callback()	接收: minidcos_if_get_service_with_cbk()
状态机: UdsTp_DiagSvc_Abstract	状态机: cross_uds_req_msg_context
超时: UdsTp_TimeOutMonitor_20ms()	超时: uds_cb_monitor_cross_domain()
模式相同: INITIAL-send, PENDING-poll	模式相同: INITIAL-send, PENDING-poll

五、完整数据流总结





六、关键差异总结

1. “主循环”不在 UdsTp 里，也不在任何函数内部的 while 里。循环由 dudsopip 线程 (dji_udsopip_task_handler → OS_Tick_Task → DCM MainFunction) 每 5-10ms 驱动，每次调用状态机走一步就返回
2. 0x22/0x2E/0x31 不走任何网络通信，直接在 SOC 本地处理 (读内存/NvM)
3. 只有 0x14/0x19/0x28/0x85 走 MiniDCOS 跨域到 MCU
4. UdsTp.c 的 UdsTp_Diag_XXSvc 是一套方案，当前产品未使用——实际使用的是 Dcm_Mcu_Proxy.c + uds_related_service.c 的组合